ContraTGT: Contrastive Pre-training and Learnable Data Augmentation of Temporal Graph Networks

Anonymous Authors

***Abstract*—Recently, there has been a surge of interest in developing specialized temporal graph neural networks and temporal transformers to adapt to the dynamism inherent in temporal networks. While these models exhibit commendable performance, they are often criticized for providing inadequate representations, particularly when faced with limited data. Contrastive learning has been introduced as a solution for graph pre-training to mitigate data scarcity. However, the data augmentation techniques on which they depend frequently lead to suboptimal augmentations, manifesting either as over-augmentation or under-augmentation. In light of these challenges, this paper delves into the under-explored domain of contrastive learning of temporal graph transformers. Specifically, we present a novel model, ContraTGT, which employs a dual-view graph transformer. This transformer is fine-tuned to extract sequences tailored for specific target nodes, encapsulating both spatial and temporal perspectives. To optimize the contrastive learning of this dual-view transformer, we put forth an innovative, learnable data augmentation method. This technique, which involves selective masking of elements within dual-view sequences, generates superior augmentations, thereby amplifying the potency of the contrastive learning approach. Extensive experiments on five public temporal network datasets demonstrate that our model can consistently outperform all baselines, especially in small training set conditions. Our code is available at https://github.com/Jaff-hi/ContraTGT.**

***Index Terms*—Graph Transformer, learnable data augmentation, temporal graphs.**

I. Introduction

Many real-world networks exhibit dynamic behavior, where interactions or edges carry associated timestamps denoting their occurrence. These are identified as temporal (or dynamic) graphs (1). Such dynamism necessitates specialized methods for temporal graph analysis, diverging from traditional graph representation learning techniques (2). Initial endeavors in embedding temporal interaction graphs predominantly yielded static representations (3; 4). A significant limitation of these approaches is their inability to accommodate newly introduced nodes or edges as graphs progress over time. To address this dynamic element and the need for time-sensitive representations, a new wave of research ushered in the concept of temporal graph neural networks (TGNNs) (5; 6), which are crafted to model both temporal nuances and structural dependencies, demonstrating adaptability to emerging nodes and edges (7; 8; 9; 10; 11; 12). A common theme among these models is the employment of neighborhood aggregation, which dynamically assimilates contextual information essential for node representation learning. Nevertheless, a potential shortcoming is the predominant focus of neighborhood aggregation on immediate contexts and recent history, which may inadvertently sideline the broader global context and invaluable long-term historical insights.

With the advent of graph transformers (13; 14; 15), a fresh perspective on representation learning in temporal networks has been established. Temporal graph transformers (TGTs) (16; 17) have been designed in alignment with traditional practices, primarily utilizing a sequence-based framework, emphasizing the sampling of extensive sequences to encompass both the global context and the historical behaviors associated with a target node. By synergizing these sampled sequences with diverse encodings, TGTs demonstrate a keen ability to identify and interpret prolonged dependencies within the temporal graph. As a testament to their efficacy, TGTs have achieved benchmark performances in a range of temporal graph learning challenges (16; 17).

The Problem. While Transformer-based approaches have proven effective, they often rely heavily on ample labeled data for model training (16; 17), which becomes a bottleneck in scenarios with restricted supervision. A promising solution lies in pre-training on graphs, which leverages the intrinsic structures of the graph through a self-supervised approach (18; 19). Within this context, contrastive learning has emerged as a potent method for pre-training on graphs, particularly for extracting underlying information (20; 21; 22). Yet, the application of contrastive learning with graph transformers on temporal graphs remains an under-explored area. This gap is especially pronounced in the realm of temporal graph representation learning under limited supervision. Motivated by this observation, our paper delves into the problem of *contrastive learning of graph transformers on temporal graphs*.

Challenges. Pre-training graph transformers on temporal graphs holds significant promise but remains an under-explored avenue. Within the domain of temporal graph representation learning, the methodology diverges significantly from the commonly explored contrastive learning that utilizes GNN backbones. Unlike these, graph transformers typically operate in a sequence-based fashion. Therefore, a typical practice is to devise a contrastive strategy tailored for sequence-based learning on temporal graphs. Yet, when considering contrastive learning on graphs, prevalent data augmentation techniques (21) like DropEdge (23) and DropNode (24) often employ random masking of raw data. However, calibrating the degree of such augmentations is intricate, potentially leading to excessive (over-augmentation) or insufficient (under-augmentation) alterations[1]. This poses a conundrum: *in the context of contrastive learning on temporal graphs, how to craft an adept contrastive strategy for sequence-based learning that harmoniously integrates an optimal augmentation technique?*

Present work. To address the aforementioned challenges, we introduce a novel framework named ContraTGT, for the Contrastive learning of Temporal Graph Transformers. At its core, ContraTGT leverages a dual-view graph transformer, which is tailored to extract sequences for a given target node, encompassing both spatial and temporal views. Such a design facilitates the assimilation of comprehensive contextual information and traces the historical evolution associated with the target node, thereby refining temporal node representation learning. In tandem with this, to proficiently pre-train the dual-view graph transformer, we architect a sequence-based contrastive strategy, which aligns seamlessly with the requirements of contrastive learning tailored for temporal graph transformers. Moreover, we introduce an adaptive, learnable data augmentation technique. By selectively masking elements within the dual-view sequences, this method crafts optimized augmentations, thereby enhancing the efficacy of the contrastive learning process.

[1]Due to space limitation, we have included a detailed empirical discussion to illustrate over-augmentation and under-augmentation in Appendix B.

We leverage dual-view node sequences sampled from temporal graphs, which are derived from both spatial and temporal perspectives, to formulate our contrastive strategy. Particularly, for each node sequence, we employ node masking—specifically, by dropping nodes within the sampled sequences, to act as the data augmentation. The foundational reason behind this approach is that the sequences, even when subjected to such masking perturbations, retain an approximation of the principal context and historical trends associated with the target node. As a result, these modified sequences can effectively function as augmented counterparts in the contrastive learning process.

While random masking serves its purpose, as highlighted earlier, it lacks precision in controlling the extent of augmentation, which can lead to over-augmentation or under-augmentation, potentially compromising the quality of contrastive learning. Therefore, we introduce a learnable data augmentation mechanism, designed to selectively mask nodes within the dual-view sequences. To steer this learnable data augmentation, we define two essential metrics: *consistency* and *diversity*. The former, consistency, gauges the prediction alignment between augmented and raw data. A higher consistency suggests that the augmentation closely mirrors the key features of the raw data, leading to aligned predictions. Conversely, diversity evaluates the variance between the augmented representation and its raw counterpart. A greater diversity indicates a more desirable augmentation. This rationale stems from the idea that, beyond capturing primary behaviors, diversity encompasses supplementary or secondary data features—essentially, the minor perturbations. These perturbations help to amplify the distinctions between augmented and raw data, aiding in the label propagation from labeled to unlabeled data. Collectively, these metrics allow for meticulous control over the noise introduced, ensuring optimal augmentations tailored for contrastive learning in temporal graphs.

Contributions. In summary, our contributions can be outlined as follows. We tackle the novel research problem of contrastive learning for graph transformers applied to temporal graphs, which represents a significant and under-explored domain. To address this challenge, we introduce a novel model ContraTGT, designed specifically for the contrastive learning of temporal graph transformers. ContraTGT not only samples sequences from both spatial and temporal perspectives but also incorporates a learnable data augmentation mechanism to generate optimized augmentations for these sequences. We conduct extensive experiments on five real-world datasets, showcasing the superior performance of ContraTGT when compared to state-of-the-art baseline methods.

II. Related Work

Graph representation learning. The development of graph embedding methods (25; 26; 27; 28) has significantly advanced the field of graph analysis. These methods aim to capture the connectivity structure of graphs while mapping nodes or substructures into low-dimensional vector spaces.

However, real-world graph networks are often dynamic, leading to increased attention on Dynamic Graph Embeddings (DGEs) (29; 30; 31). These methods build upon static graph embedding techniques but incorporate temporal information. DGEs can be categorized into two main types: discrete-time dynamic graph embeddings and continuous-time dynamic graph embeddings. In the discrete-time setting, many DGE methods learn embeddings for each snapshot and ensure smooth transitions between consecutive snapshots (32; 9; 33; 34). On the other hand, continuous-time methods (12; 35; 36; 37; 38) manipulate dynamic graphs directly, offering fine-grained control over temporal graph representation learning.

Recently, the capabilities of fully connected global attention mechanisms have prompted researchers to explore the application of Transformers (39) for graph representation learning (15; 14). However, most existing models require the entire graph as input, limiting their scalability due to the high computational cost of full attention (40). Many modern graph transformers operate in a sequence-based manner, allowing them to capture contextual information effectively within the graph (16; 41; 17).

Model pre-training on graphs. Graph pre-training (18; 20) is a valuable technique in deep learning, enabling models to learn useful features from input graphs in a self-supervised manner. This approach results in universal models capable of performing various tasks on graphs. Graph contrastive learning (42; 11; 21; 43) is a prevalent method for improving perturbation invariance through graph augmentation.

However, the challenges lie in determining how to perform effective augmentation and evaluating the quality of augmented data. Contrastive learning with transformers on temporal graphs remains relatively unexplored. DyFormer (17), for instance, employs cross-view contrastive learning, contrasting sequences from dual perspectives. Nevertheless, its reliance on random sequence sampling and preset weightings can lead to over-augmentation or under-augmentation, potentially compromising data quality (44). Another work TCL (16) employs two types of time encodings, including depth embedding and time embedding, to fuse with the context of the target node as the input of the graph Transformer. It employs contrastive learning in a plain way, contrasting the positive pair (a true link) and the negative pair (a negative link). In contrast, the temporal trend on the graph is not considered by TCL. We also devise the advanced learnable data augmentation under the guidance of consistency and diversity for self-supervised learning.

Other graph augmentation approaches. Some graph contrastive learning approaches utilize advanced augmentation approaches (45; 46). JOAO (45) operates within an adversarial training framework, assuming that effective augmentations result in high contrastive learning loss, overlooking consistency. Its learning objective involves selecting the optimal augmentation pair from a predefined pool of augmentation types for graph contrastive learning, requiring domain knowledge to manually construct and configure the augmentation pool for selection. Another work GCA (46) relies on the assumption that a good augmentation should discard unimportant edges and features while retaining important ones. The adaptive augmentation module in GCA, when applied to an input graph, remains static and non-learnable throughout training. This static nature arises from the reliance on node centrality scores, resulting in unchanging dropping rates for edges and node features.

III. Preliminaries

Graph. A graph can be denoted as $G = \{V, E, \mathbf{X}\}$, where V is the set of nodes, E is the set of edges, and $\mathbf{X} \in \mathbb{R}^{|V| \times d_X}$ is the node feature matrix. Let $\mathbf{x}_v \in \mathbb{R}^{d_X}$ denote the feature vector of node v.

Temporal graph. A temporal graph integrates temporal details into its structure. Formally, a temporal graph can be represented as $G = \{V, E, \mathbf{X}, T, \phi, \varphi\}$. Here, V and E denote the nodes and edges, respectively, that exist within the temporal graph for any specified time $t \in T$. The function ϕ serves as a node presence indicator $\phi : (V, T) \to \{0, 1\}$, illustrating if a node $v \in V$ is part of the graph at time $t \in T$. Similarly, φ operates as an edge presence indicator, defined by $\varphi : (E, T) \to \{0, 1\}$ indicating the presence of an edge $e \in E$ in the graph at time $t \in T$.

Problem formulation. Given a temporal graph G, a specific timestamp t, and a node v, our objective is to derive a meaningful

representation for the node v at the timestamp t, *i.e.*, $\mathbf{h}_{(v,t)}$, by harnessing both its contextual and historical information. Specifically, our primary attention in this paper is centered on the dynamic link prediction task. When presented with a tuple (v, u, t)—indicating a link between nodes v and u at timestamp t, we aim to predict its existence based on the representations of nodes v and u, denoted as $\mathbf{h}_v$ and $\mathbf{h}_u$, respectively, which then underpins our primary objective. Formally, the probability of the existence of this interaction, represented as $p^t_{(v,u)}$, is determined by

$$p^t_{(v,u)} = P(v, u, t) = \text{MLP}(\mathbf{h}_v \parallel \mathbf{h}_u), \tag{1}$$

where $\cdot \parallel \cdot$ symbolizes the concatenation operation and $\text{MLP}(\cdot)$ denotes a fully connected layer. With this probability in hand and given a set of tuples that constitute our training data, denoted as T_{tr}, the loss function can be articulated using binary cross-entropy:

$$\mathcal{L}_{\text{tr}} = -\sum_{(v,u,t)\in T_{\text{tr}}} \log p^t_{(v,u)} + \mathbb{E}_{x\sim\mathcal{U}_v} \log(1 - p^t_{(v,x)}), \tag{2}$$

where $\mathcal{U}_v$ represents a uniform distribution over all nodes for the purpose of negative sampling, excluding the node v. For the sake of brevity, in subsequent sections, we might represent the node representation without the timestamp t as $\mathbf{h}_v$.

Note that, data labeling often entails a significant investment of effort. Therefore, our research predominantly operates under conditions of *limited supervision*. To mitigate this, we advocate the use of contrastive learning on temporal graphs, which can efficiently extract the inherent structural nuances of the graph, fostering representation learning in a self-supervised fashion.

Graph Transformers. In the conventional Transformer architecture (39), two primary components stand out: multi-head self-attention and the feed-forward network. The self-attention mechanism is designed to gauge the relationship between the focal node and its contextual counterparts, especially nodes within the same sequence. For clarity, in layer l, assuming a single attention head and given an input sequence representation $\mathbf{H}^{(l-1)} = [\mathbf{h}_0^{(l-1)}, \mathbf{h}_1^{(l-1)}, ..., \mathbf{h}_{n-1}^{(l-1)}] \in \mathbb{R}^{n\times d_{l-1}}$, where d_{l-1} is the representation dimension in layer $l-1$ and $\mathbf{h}_i^{(l-1)} \in \mathbb{R}^{d_{l-1}}$ is the hidden state at position i. The self-attention initially converts the sequence representation $\mathbf{H}^{(l-1)}$ into the query, key, and value representations, *i.e.*, $\mathbf{Q}^{(l)}, \mathbf{K}^{(l)}, \mathbf{V}^{(l)}$, by employing three parameter matrices $\mathbf{W}_Q^{(l)} \in \mathbb{R}^{d_{l-1}\times d_l^K}$, $\mathbf{W}_K^{(l)} \in \mathbb{R}^{d_{l-1}\times d_l^K}$ and $\mathbf{W}_V^{(l)} \in \mathbb{R}^{d_{l-1}\times d_l^V}$, as

$$\mathbf{Q}^{(l)} = \mathbf{H}^{(l-1)}\mathbf{W}_Q^{(l)},\ \mathbf{K}^{(l)} = \mathbf{H}^{(l-1)}\mathbf{W}_K^{(l)},\ \mathbf{V}^{(l)} = \mathbf{H}^{(l-1)}\mathbf{W}_V^{(l)}. \tag{3}$$

Subsequently, the attentive output can be computed as

$$\text{ATT}(\mathbf{H}^{(l-1)}) = \text{SOFTMAX}(\mathbf{Q}^{(l)}\mathbf{K}^{(l)\top}/\sqrt{d_l^K})\mathbf{V}^{(l)}. \tag{4}$$

Typically, this intermediate representation undergoes further processing through feed-forward layers, given by

$$\mathbf{H}^{(l)} = \text{LN}\big(\text{FFN}\big(\text{LN}(\text{ATT}(\mathbf{H}^{(l-1)}) + \mathbf{H}^{(l-1)})\big) + \widetilde{\mathbf{H}}^{(l)}\big), \tag{5}$$

where $\text{FFN}(\cdot)$ refers to the feed forward layer, and $\text{LN}(\cdot)$ signifies layer normalization (47). By iteratively stacking L such layers, Transformers discern relationships across various positions in the input sequences. The resultant output $\mathbf{H}^{(L)} \in \mathbb{R}^{n\times d}$ serves as the sequence representations for subsequent tasks. We utilize θ_t to represent the parameters set in the Transformer.

IV. The Proposed Model: ContraTGT

In this section, we delve into the illustrations of our proposed model, ContraTGT, as depicted in Fig. 1. As shown in Fig. 1(b), starting with an input temporal graph, we initiate with dual-view sequence sampling from both spatial and temporal perspectives as explained in Sect. IV-A. This is followed by the pre-training of the Transformer using contrastive learning (Fig. 1(c)), as outlined in Sect. IV-B, leveraging our unique learnable data augmentation to generate optimal augmentations. Concluding the process, we utilize the pre-trained model for further fine-tuning (Fig. 1(d)), as detailed in Sect. IV-C, applying the limited supervision provided to execute predictions for downstream tasks, such as link prediction.

A. Dual-View Sequence Sampling

Leveraging the self-attention mechanism, Transformers adeptly discern global relationships between object pairs (39). This capability has been effectively extended to graph-level representation learning (15; 14; 48; 49). However, a significant drawback of this method lies in its high memory consumption, rendering it impractical for large-scale graphs. In response to this challenge, prevalent techniques, particularly for node-level representation learning on expansive graphs, commonly utilize sequence sampling methods (50; 51). Specifically, a node sequence surrounding the target node is sampled to capture its local context, which is then processed by the Transformer to derive node representations.

To holistically capture the dynamic behaviors of nodes within temporal graphs, we draw inspiration from (10). Our approach involves sequence sampling from both spatial and temporal views, thereby portraying both contextual nuances and historical developments. These sampled sequences are then funneled into the Transformer, facilitating comprehensive node representation learning.

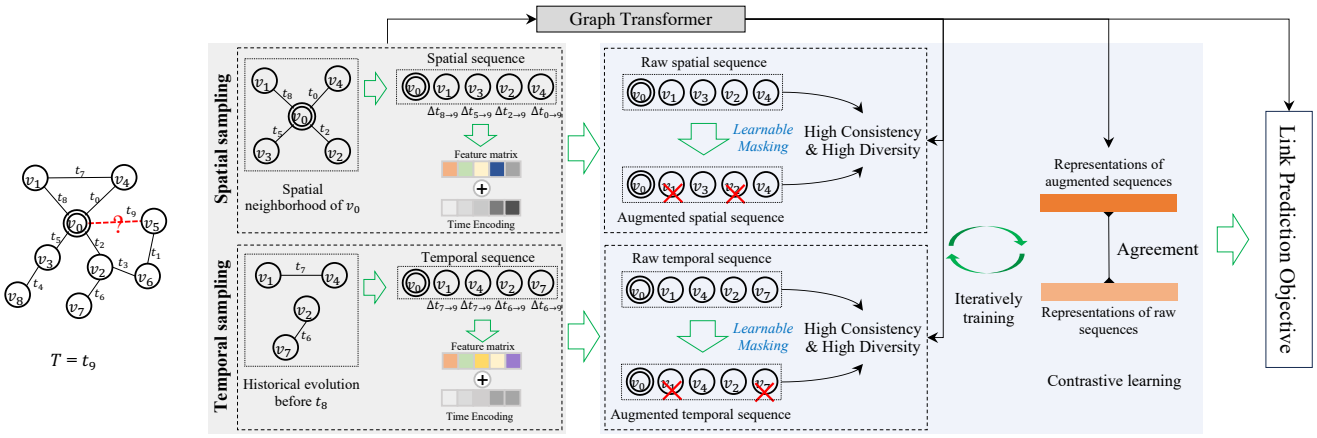
Spatial Sampling. Spatial sampling is primarily oriented towards capturing a sequence of neighboring nodes with length l_s surrounding a designated target node, thereby providing a rich depiction of its local context. Specifically, for contextually neighboring nodes, they are arranged in descending order based on the timestamps of their interactions with the target node. This is because recent interactions, being closer in the temporal dimension, might exert a more pronounced influence on the target node.

For illustrative clarity, consider Fig. 1. Taking the target node v_0 as an example, its contextual neighborhood can be represented as $\mathcal{N}_{v_0} = \{v_1, v_2, v_3, v_4\}$ (Fig. 1(b) (top)). Among these neighbors connected to v_0, arranging them in descending order by their interaction timestamps results in the sequence v_1, v_3, v_2, and v_4. Therefore, we form the spatial sequence as $(v_0, v_1, v_3, v_2, v_4)$. Subsequently, for each node in the sequence, time encoding is derived by computing the time interval spanning from the current timestamp back to the timestamp of the specific interaction.

As a case in point, as illustrated in Fig. 1(b) (top), the time interval associated with the neighboring node v_1 is determined by $t_9 - t_8$, denoted as $\Delta t_{8\to 9}$. Following the computation of these time intervals, the subsequent step involves encoding time intervals as described in (10). For a more nuanced understanding of time encoding, please refer to Appendix A. To culminate this process, the concatenated vector—comprising the feature vector and the time encoding for each node—becomes the input feature vector for the Transformer.

Temporal Sampling. Temporal sampling seeks to capture the global dynamics preceding the most recent interaction of a target node, with length l_t. Specifically, this sampling method aims to unravel the dynamic tendencies exhibited by the temporal graph over the latest time frame. Understanding these tendencies is paramount for representation learning of the target node, since the most recent interaction of the target node tends to adhere to this evolving temporal pattern.

For example, as shown in Fig. 1(a), the most recent interaction involving the target node v_0 is its engagement with v_1 at timestamp t_8. In this context, our approach zeroes in on the k interactions



(a) Input temporal graph (b) Dual-view sequence sampling (c) Pre-training with contrastive learning (d) Supervised loss

Fig. 1: The overall framework of ContraTGT. In the input temporal graph, the timestamp t_i is in an increasing order, *i.e.*, $t_i < t_j$ if $i < j$. For target node v_0, at timestamp t_9, the goal is to predict whether there exists a link between v_0 and v_5.

immediately preceding t_8. As an example, with $k = 2$, the interactions of interest are (v_1, v_4, t_4) and (v_2, v_7, t_6). The terminal nodes of these interactions are used to forge the temporal sequence, denoted as $(v_0, v_1, v_4, v_2, v_7)$. Notably, nodes within this sequence are chronologically arranged in descending order based on their associated timestamps. In parallel fashion to spatial sampling, each node within this temporal sequence has a corresponding time interval calculated. This interval spans from the current timestamp (*e.g.*, t_9) backward to the timestamp marking the specific interaction, as delineated in Fig. 1(b) (bottom). The established time encoding mechanism is subsequently employed, converting this time interval into a time-encoded format. This encoded value is then concatenated with the node feature vectors, thereby preparing the composite vector as the input for the Transformer.

Discussion. The adoption of dual-view sequences in our approach enables a more nuanced depiction of both contextual nuances and historical evolution, thus enriching node representation learning via the transformer. This design choice distinctly sets our methodology apart from existing works. An exemplar in this domain, DyFormer (17), operates based on a discrete snapshot paradigm. Within this framework, the spatial encoding is deduced from the shortest distance between nodes within a given snapshot. Concurrently, the temporal encoding arises from the presence of edges in that snapshot. These encodings subsequently inform the input of the transformer. Our proposed model, in contrast, operates within a continuous-time framework. Specifically, we lean on the contextual neighborhood surrounding a target node when sampling for the spatial sequence. Moreover, our temporal sequence sampling is anchored on the historical trajectory leading up to the most recent interaction of the target node. Such a design allows our model to more accurately capture both the intricate structural details inherent to the local context (from a spatial view) and the historical trends that the target node has participated in (from a temporal view). Consequently, our model offers a more holistic and granular understanding compared to existing methodologies.

B. Pre-Training with Contrastive Learning

Despite the effectiveness of graph contrastive learning, the nuances of contrastive learning in temporal graphs remain relatively uncharted. An associated study, DyFormer (17), employs cross-view contrastive learning, contrasting sequences drawn from dual perspectives. However, its reliance on random sequence sampling, contingent upon pre-set weightings, potentially leads to over- or under-augmentation, compromising data integrity (44).

To navigate this challenge, we harness the potential of dual-view sequence sampling and introduce an innovative contrastive learning strategy centered around learnable data augmentation, poised to optimize augmentation generation. In the subsequent part, we begin by introducing the essential concepts of consistency and diversity, forming the basis for our learnable data augmentation approach, which we will then provide a detailed illustration. Finally, we formalize our contrastive learning strategy to complete the model description.

Consistency and Diversity. To adeptly guide the data augmentation module in creating ideal data augmentations, we introduce two metrics—consistency and diversity, inspired by (44). These metrics offer a quantitative lens to evaluate the relationship between the original data and its augmented counterpart. While consistency underscores the importance of preserving identical predictions, diversity highlights the need for variation in representation distributions. This dual-metric paradigm crafts augmentations that not only align with the same prediction but also span diverse distribution attributes, ultimately paving the way for label propagation across a broader space populated with nuanced features. Collectively, these metrics steer the optimization process of our learnable data augmentation towards generating superior augmentations.

Given a data augmentation method, $\Psi(\cdot;\theta_\Psi)$, parameterized by θ_Ψ, an augmentation of a raw object x can be represented as $\hat{x} = \Psi(x;\theta_\Psi)$. Employing an encoder $\delta(\cdot;\theta_\delta)$, both x and $\hat{x}$ can be mapped into their low-dimensional representations, *i.e.*, $\mathbf{h}_x = \delta(x;\theta_\delta)$ and $\mathbf{h}_{\hat{x}} = \delta(\hat{x};\theta_\delta)$. On the one hand, consistency is manifested when the prediction outcomes for augmented data align seamlessly with those of the raw data. Given a prediction function, $f(\cdot)$, like a classification or link predictor, the consistency between object x and its augmentation $\hat{x}$ can be formalized as

$$C(x,\hat{x}) = \mathbb{I}\big(f(\mathbf{h}_x) = f(\mathbf{h}_{\hat{x}})\big), \tag{6}$$

where $\mathbb{I}(\cdot)$ is an indicator function, returning 1 if the condition is met and 0 otherwise. Differently, diversity captures the variance in representations between the original object and its augmentation. Formally, the diversity between x and $\hat{x}$ can be formalized as

$$D(x,\hat{x}) = d(\mathbf{h}_x, \mathbf{h}_{\hat{x}}), \tag{7}$$

where $d(\cdot,\cdot)$ is a distance function, the mainstream approach uses the Euclidean distance to account for variance in representations(52). Ideally, a proficient augmentation methodology would produce augmented samples that strike a balance between high consistency with original data and substantial inherent diversity.

Learnable data augmentation. Leveraging the capabilities of our transformer model, which is rooted in the dual-view sequences for representation learning, we introduce a data augmentation paradigm tailored to these sequences, specifically for object masking. Consider, for example, a raw spatial sequence as depicted in Fig. 1(c) (left) that follows the order $(v_0, v_1, v_3, v_2, v_4)$. Our objective is to mask the nodes, like v_1 and v_2, resulting in the augmented sequence (v_0, v_3, v_4). This augmentation approach offers multiple benefits. (1) While nodes are masked, the primary sequential order remains intact, ensuring the retention of the fundamental characteristics of the sequence. (2) The simplicity of this masking process paves the way for incorporating learnable elements into our data augmentation method.

Proceeding with this line of thinking, we aim to make the masking procedure inherently learnable. Specifically, given an initial sequence $s = (v_0, v_1, \ldots, v_{n-1})$ associated with its feature sequence $s_x = (\mathbf{x}_{v_0}, \mathbf{x}_{v_1}, \ldots, \mathbf{x}_{v_{n-1}})$, we engage the transformer to encode the sequence s, producing a corresponding representation sequence, denoted as $S = (\mathbf{h}_{v_0}, \mathbf{h}_{v_1}, \ldots, \mathbf{h}_{v_{n-1}})$.

Subsequent to obtaining the representation sequence S, we introduce a learnable data augmentation module, enabling the synthesis of its augmented counterpart, $\hat{s}$. This process leverages learnable masking to selectively retain objects that manifest high consistency and diversity, while concurrently discarding the remainder. Note that the primary objective of learnable data augmentation is to generate an augmented sequence that will subsequently be used to compute new representations within the transformer. For the sake of simplicity, we augment the feature sequence $\hat{s}_x$ as follows:

$$\hat{s}_x = \Psi_{lm}(S; \theta_{\Psi_{lm}}), \tag{8}$$

where $\Psi_{lm}(\cdot; \theta_{\Psi_{lm}})$, parameterized by $\theta_{\Psi_{lm}}$, represents the learnable masking module, generating the augmented sequence for an input sequence. To astutely discern which objects to retain and which to mask, the attention mechanism becomes indispensable. Applied to the input representation sequence S, this mechanism calculates a weight for each position in S. These weights are then crisply dichotomized into either 0's or 1's, as guided by a pre-established masking rate ρ. Function $\Psi_{lm}(\cdot; \theta_{\Psi_{lm}})$ can be detailed as:

$$\hat{s}_x = \Psi_{lm}(S; \theta_{\Psi_{lm}}) = \textsc{SharPen}(\textsc{Atten}(S, v); \rho) \odot s_x. \tag{9}$$

The function $\textsc{Atten}(S, v)$ symbolizes the attention mechanism. It processes each representation in S, yielding a vector of n-dimensional vector, which underscores the significance of every object in s_x. A distilled representation of this attention module is

$$\textsc{Atten}(S, v) = \Big[\frac{\exp(\mathbf{W}\mathbf{h}_v \cdot \mathbf{W}\mathbf{h}_{v_i})}{\sum_{v_j \in s} \exp(\mathbf{W}\mathbf{h}_v \cdot \mathbf{W}\mathbf{h}_{v_j})}\Big]_{i=0}^{n-1}, \tag{10}$$

where $\mathbf{W} \in \mathbb{R}^{d \times d}$ is a parameter matrix. The notation $[f(i)]_{i=0}^{n-1}$ denotes an n-dimensional vector, where each dimension i reflects the output of the function $f(i)$. Function $\textsc{SharPen}(\mathbf{h}; \rho)$ is devised to emphasize positions with the most salient values in $\mathbf{h}$ as ones, while relegating others to zeros. The proportion of zeros is governed by ratio ρ. For clarity, consider the vector $\mathbf{a} = [0.2, 0.3, 0.4, 0.1]$ with $\rho = 0.5$; this would translate to $\textsc{SharPen}(\mathbf{a}; \rho) = [0, 1, 1, 0]$. A salient aspect of our design is that the sharpen function is gradient-preserving, guaranteeing seamless backpropagation. This fosters end-to-end model training by driving the weights closer to definitive ones and zeros, rather than rigidly enforcing them. Detailed pseudocode for the sharpen function is in the subsection IV-D.

To enable learnable augmentation, the augmented representations—obtained by inputting s_x into the transformer—are employed to compute the consistency and diversity losses, strictly adhering to the strategies delineated in Equations (6) and (7). In the pre-training phase, we adopt link prediction as the target prediction function. Consider an extant link (v, u)[2]. Given the representations of nodes v and u, represented as $\mathbf{h}_v$ and $\mathbf{h}_u$[3], along with the representation of the augmented node $\hat{v}$[4] as $\mathbf{h}_{\hat{v}}$, we commence by determining the link prediction probability for the pair (v, u) as $p_{(v,u)}$ according to Eq. (1), as well as the link prediction probability for pair $(\hat{v}, u)$ as $p_{(\hat{v},u)}$. Using the pre-training link set L_{pt}, both the pristine and augmented predictions are harnessed to derive the consistency and diversity losses, designated $\mathcal{L}_{\Psi c}$ and $\mathcal{L}_{\Psi d}$, respectively:

$$\begin{aligned}\mathcal{L}_{\Psi c} = -\sum\nolimits_{(v,u)\in L_{\text{pt}}} (p_{(v,u)} \log p_{(\hat{v},u)} \\ + (1 - p_{(v,u)}) \log(1 - p_{(\hat{v},u)})),\end{aligned} \tag{11}$$

$$\mathcal{L}_{\Psi d} = -\sum\nolimits_{(v,u)\in L_{\text{pt}}} d(\mathbf{h}_v, \mathbf{h}_{\hat{v}}). \tag{12}$$

In the concluding step, the two losses are amalgamated with a weighting factor α, *i.e.*,

$$\mathcal{L}_{\text{aug}} = \mathcal{L}_{\Psi c} + \alpha \mathcal{L}_{\Psi d}. \tag{13}$$

This weighted sum steers the training of the learnable data augmentation, a process visually elucidated in Fig. 1(c).

Contrastive learning. Contrastive learning fundamentally seeks to ensure that representations of raw data and its corresponding augmentations are consistent when subjected to suitable transformations. As discussed earlier, we derived augmented data exhibiting both high consistency and diversity using our learnable data augmentation approach. Conceptually, for each node v within the pre-training nodes set V_{pt}, we randomly select another node u as a negative instance. Our contrastive objective prioritizes enhancing the agreement between v and its augmentation $\hat{v}$ while simultaneously diminishing the concurrence between v and the negative instance u. Given the representations—specifically $\mathbf{h}_v$, $\mathbf{h}_{\hat{v}}$, and $\mathbf{h}_u$—the contrastive loss is articulated as:

$$\mathcal{L}_{\text{con}} = -\sum\nolimits_{v\in V_{\text{pt}}, u\in V_{\text{pt}}\setminus v} \log \frac{\exp(\mathbf{h}_v \cdot \mathbf{h}_{\hat{v}})}{\exp(\mathbf{h}_v \cdot \mathbf{h}_{\hat{v}}) + \exp(\mathbf{h}_v \cdot \mathbf{h}_u)}. \tag{14}$$

Note that, in each iteration, both the augmentation loss $\mathcal{L}_{\text{aug}}$ and the contrastive loss $\mathcal{L}_{\text{con}}$ are optimized in an alternating fashion. Specifically, during the optimization of $\mathcal{L}_{\text{aug}}$, the parameters of the transformer module are fixed, focusing solely on fine-tuning the parameters in the learnable data augmentation module, *i.e.*, $\theta_{\Psi_{lm}}$, ensuring the generation of optimal augmentations. In contrast, while optimizing $\mathcal{L}_{\text{con}}$, we fix the parameters of the learnable data augmentation module and only optimize the parameters of the transformer, *i.e.*, θ_t, aiming to hone an efficient transformer.

C. Objective

Considering the proposed dual-view sequences, we proceed by associating node input features with their respective time encodings

[2]Here, we omit the timestamp for simplicity.

[3]This representation might be derived from either the spatial or temporal view, as the consistency and diversity losses are computed independently for each view.

[4]Notably, the augmented entity pertains to the sequence associated with node v, rather than the node itself. That is, we transform the feature sequence of node v, termed $s_{x,v}$, into its augmented counterpart $\hat{s}_{x,v}$. This augmented sequence then informs the representation of node $\hat{v}$, denoted as $\mathbf{h}_{\hat{v}}$.

for each view, which are then utilized as inputs for the transformer. Formally, within the initial transformer layer, we compute its input matrix as $\tilde{\mathbf{H}}^{(0)} = \mathbf{H}^{(0)} + \mathbf{H}^t$, where $\mathbf{H}^{(0)}$ can be initialized as node features, and $\mathbf{H}^t$ denotes the time encodings. Following the separate encoding of each view by the transformer, in the final layer, we obtain their respective representations denoted as $\mathbf{H}_s$ and $\mathbf{H}_t$ (we omit the layer superscript here) for the spatial and temporal views, respectively. These representations are concatenated to form the node v representation, denoted as $\mathbf{h}_v = \mathbf{H}_s[v] \parallel \mathbf{H}_t[v]$, which will be utilized in downstream tasks.

With these node representations in hand, we can formalize the objective in the fine-tuning step, as denoted by $\mathcal{L}_{\text{tr}}$ in Eq. (2). During this step, we focus on optimizing the parameters within the Transformer module, *i.e.*, θ_t.

D. Algorithms and Complexity Analysis

Pre-training paradigm. We present the algorithm for the pre-training of our proposed model ContraTGT in Algorithm 1. Note that Algorithm 2, which represents the node representation learning function, is utilized within Algorithm 1 (lines 5, 6, and 13).

Within Algorithm 1, a batch of interactions is sampled as outlined in line 3. The subsequent lines, 4-14, are dedicated to computing both the augmentation loss (specified in line 11) and the contrastive loss (detailed in line 14). Specifically, lines 5-6 focus on deriving the representations for the selected nodes along with their augmentations. Following this, their link predictions are ascertained in lines 7-8. The consistency and diversity loss are calculated in lines 9 and 10, respectively. Lines 12-13 involve sampling a negative node for which we then calculate its representation. To optimize our model, we address the augmentation loss by freezing the transformer, as described in line 15. Thereafter, we minimize the contrastive loss by freezing the learnable data augmentation module, as indicated in line 16.

Complexity analysis. Overall, the model pre-training in Algorithm 1 consists of two main components: the optimization of learnable data augmentation (*i.e.*, $\mathcal{L}_{\text{aug}}$) and the optimization of the Transformer (*i.e.*, $\mathcal{L}_{\text{con}}$), which are conducted iteratively. Let d denote the representation dimension. On one hand, for the optimization of learnable data augmentation, the representation learning for each node via the Transformer (assuming one layer) involves a complexity of $O(B(l_s^2 d + l_t^2 d))$, where l_s and l_t denote the length of spatial and temporal sequences, respectively. For data augmentation, its calculation only involves one traversal of both spatial and temporal sequences, with a complexity of $O(B(l_s d + l_t d))$. Therefore, the overall complexity of optimizing learnable data augmentation is $O(B(l_s^2 d + l_t^2 d))$. On the other hand, the optimization of the Transformer involves a similar complexity of $O(B(l_s^2 d + l_t^2 d))$, which comes from the calculation of the representation of the negative node u. In summary, the complexity of the pre-training of ContraTGT is generally $O(B(l_s^2 d + l_t^2 d))$, which is proportional to the batch size and hidden dimension, and quadratically proportional to the length of spatial and temporal sequences. Note that in our implementation, the length of spatial and temporal sequences are constants, so the time cost will linearly increase as the graph size increases, which is also empirically demonstrated in the Scalability test (Section V-C).

The sharpen function. We provide the pseudocode for the sharpen function in Alg. 3.

V. Experiments

In this section, we conduct extensive experiments to evaluate the effectiveness of ContraTGTon five benchmark datasets.

Algorithm 1 Pre-Training of ContraTGT

Input: A temporal graph $G = \{V, E, \mathbf{X}, T\}$, length of spatial sampling l_s, length of temporal sampling l_t,
Output: Parameters θ_t.

1: **while** not converged **do** ▷ Pre-training iteration
2: $\mathcal{L}_{\text{aug}} \leftarrow 0$ and $\mathcal{L}_{\text{con}} \leftarrow 0$;
3: Sample a batch of interactions $B \in G$;
4: **for** each interaction $(v, u, t) \in B$ **do** ▷ Sample an interaction
5: $\mathbf{h}_v$, $\mathbf{h}_{\hat{v}} = \text{NRL}(v)$; ▷ Node representation learning
6: $\mathbf{h}_u$, $\mathbf{h}_{\hat{u}} = \text{NRL}(u)$; ▷ Node representation learning
7: $p_{(v,u)} = \text{MLP}(\mathbf{h}_v \parallel \mathbf{h}_u)$; ▷ Eq. (1)
8: $p_{(\hat{v},u)} = \text{MLP}(\mathbf{h}_{\hat{v}} \parallel \mathbf{h}_u)$; ▷ Eq.(1)
9: $\mathcal{L}_{\Psi c} = -p_{(v,u)} \log p_{(\hat{v},u)} + (1 - p_{(v,u)}) \log(1 - p_{(\hat{v},u)})$; ▷ Consistency loss, Eq. (11)
10: $\mathcal{L}_{\Psi d} = -d(\mathbf{h}_v, \mathbf{h}_{\hat{v}})$; ▷ Diversity loss, Eq. (12)
11: $\mathcal{L}_{\text{aug}} \leftarrow \mathcal{L}_{\text{aug}} + \mathcal{L}_{\Psi c} + \alpha \mathcal{L}_{\Psi d}$; ▷ Augmentation loss, Eq. (13)
12: Randomly sample a negative node q;
13: $\mathbf{h}_q$, $\mathbf{h}_{\hat{q}} = \text{NRL}(q)$; ▷ Node representation learning
14: $\mathcal{L}_{\text{con}} \leftarrow \mathcal{L}_{\text{con}} - \log \frac{\exp(\mathbf{h}_v \cdot \mathbf{h}_{\hat{v}})}{\exp(\mathbf{h}_v \cdot \mathbf{h}_{\hat{v}}) + \exp(\mathbf{h}_v \cdot \mathbf{h}_u)}$; ▷ Eq. (14)
15: Minimize $\mathcal{L}_{\text{aug}}$ to optimize $\theta_{\Psi_{lm}}$ while fixing θ_t;
16: Minimize $\mathcal{L}_{\text{con}}$ to optimize θ_t while fixing $\theta_{\Psi_{lm}}$;
17: **return** θ_t, $\theta_{\Psi_{lm}}$.

Algorithm 2 Node Representation Learning (NRL)

Input: Node v;
Output: $\mathbf{h}_v$ and $\mathbf{h}_{\hat{v}}$ (representations of v and its augmentation $\hat{v}$).

1: $s_v^s \leftarrow$ Spatial Sampling for v; ▷ Spatial sampling
2: $s_v^t \leftarrow$ Temporal Sampling for v; ▷ Temporal sampling
3: $S_v^s \leftarrow \big(\text{TF}(s_u^s; \theta_t)\big)_{u \in s_v^s}$; ▷ Representation seq., Eq. (5)
4: $S_v^t \leftarrow \big(\text{TF}(s_u^t; \theta_t)\big)_{u \in s_v^t}$; ▷ Representation seq., Eq. (5)
5: $\hat{s}_v^s, \hat{s}_v^t \leftarrow \Psi_{lm}(S_v^s; \theta_{\Psi_{lm}}), \Psi_{lm}(S_v^t; \theta_{\Psi_{lm}})$; ▷ Augmentation, Eq. (9)
6: $\mathbf{h}_v^s$, $\mathbf{h}_v^t = \text{TF}(s_v^s; \theta_t), \text{TF}(s_v^t; \theta_t)$; ▷ Representation node, Eq. (5)
7: $\mathbf{h}_{\hat{v}}^s$, $\mathbf{h}_{\hat{v}}^t = \text{TF}(s_{\hat{v}}^s; \theta_t), \text{TF}(s_{\hat{v}}^t; \theta_t)$; ▷ Representation node, Eq. (5)
8: $\mathbf{h}_v = \mathbf{h}_v^s \parallel \hat{s}_v^t$, $\mathbf{h}_{\hat{v}} = \mathbf{h}_{\hat{v}}^s \parallel \mathbf{h}_{\hat{v}}^t$; ▷ Concatenation
9: **return** $\mathbf{h}_v$, $\mathbf{h}_{\hat{v}}$.

A. Experimental Setup

1) Datasets: We evaluate our approach using five publicly available benchmark temporal networks.

We present the dataset details below. In particular, Table I also provides insights into the ratio of nodes that share multiple interactions (edges), where "-" denotes that the statistics is not provided by the authors.

- **soc-wiki-elec**[5]: The network presents the Wikipedia administrator election and voting history, which contains more than 7 thousand users and 107 thousand interactions.
- **ia-slashdot-reply-dir**[6]: This is a reply network collected from the technology website Slashdot over 977 days. The network has the largest nodes and edges for future link prediction, consisting of 51 thousand users and over 140 thousand replies.

[5]https://networkrepository.com/soc-wiki-elec.php
[6]https://networkrepository.com/ia-slashdot-reply-dir.php

TABLE I: Summary of datasets.

	# Nodes	# Interactions	# Repetition
soc-wiki-elec	7,118	107,079	4.2%
ia-slashdot-reply-dir	51,083	140,778	0.2%
soc-sign-bitcoinotc	5,881	35,592	0.0%
Social Evolve 1_month	74	176,088	-
Ubuntu	159,316	964,437	-

TABLE II: Results of small training set(10%) link prediction task (in percent with std; best bolded and runner-up underlined).

Methods	soc-wiki-elec			ia-slashdot-reply-dir			soc-sign- bitcoinotc			Social E. 1 m.			ubuntu		
	AUC	AP	Acc	AUC	AP	Acc	AUC	AP	Acc	AUC	AP	Acc	AUC	AP	Acc
GraphSAGE	0.589±0.017	0.588±0.011	0.556±0.009	0.600±0.008	0.596±0.012	0.576±0.011	0.577±0.005	0.575±0.009	0.533±0.013	0.540±0.009	0.540±0.012	0.516±0.014	0.544±0.013	0.531±0.008	0.515±0.011
JODIE	0.548±0.007	0.543±0.006	0.532±0.007	0.663±0.022	0.659±0.021	0.620±0.018	0.680±0.013	0.690±0.016	0.632±0.017	0.764±0.003	0.692±0.003	0.722±0.005	0.774±0.017	0.810±0.008	0.695±0.006
CAWN	0.788±0.013	0.743±0.019	0.761±0.007	0.892±0.008	0.904±0.010	0.801±0.010	0.831±0.004	0.866±0.003	0.741±0.012	0.871±0.030	0.875±0.003	0.721±0.005	0.774±0.018	0.811±0.008	0.695±0.006
TIP-GNN	0.747±0.009	0.606±0.020	0.63±0.027	0.809±0.031	0.634±0.034	0.651±0.037	0.843±0.010	0.706±0.011	0.775±0.011	0.841±0.003	0.702±0.009	0.773±0.005	0.795±0.010	0.814±0.006	0.807±0.008
TAP-GNN	0.952±0.008	0.952±0.010	0.924±0.022	0.902±0.001	0.877±0.007	0.806±0.021	0.801±0.008	0.774±0.025	0.731±0.016	0.771±0.003	0.673±0.002	0.731±0.003	0.847±0.004	0.846±0.008	0.812±0.005
DyRep	0.538±0.011	0.535±0.009	0.527±0.008	0.659±0.012	0.658±0.019	0.615±0.010	0.688±0.008	0.692±0.011	0.634±0.015	0.719±0.037	0.676±0.038	0.652±0.034	0.763±0.003	0.794±0.003	0.695±0.008
TGAT	0.763±0.014	0.745±0.018	0.681±0.017	0.836±0.004	0.878±0.002	0.787±0.001	0.766±0.005	0.785±0.009	0.704±0.011	0.626±0.010	0.625±0.011	0.579±0.009	0.817±0.024	0.853±0.017	0.756±0.026
APAN	0.832±0.009	0.845±0.010	0.797±0.006	0.798±0.004	0.781±0.011	0.715±0.008	0.767±0.001	0.766±0.004	0.720±0.006	0.745±0.012	0.730±0.007	0.707±0.006	0.731±0.011	0.782±0.004	0.660±0.004
TGN	0.867±0.012	0.881±0.010	0.767±0.012	0.864±0.015	0.878±0.014	0.663±0.008	0.766±0.006	0.801±0.006	0.691±0.006	0.904±0.003	0.882±0.006	0.850±0.003	0.832±0.006	0.865±0.004	0.770±0.004
TIGER	0.832±0.011	0.845±0.014	0.697±0.009	0.942±0.005	0.949±0.002	0.805±0.006	0.763±0.011	0.806±0.008	0.694±0.014	0.925±0.002	0.909±0.004	0.887±0.006	0.831±0.005	0.882±0.003	0.610±0.012
FTM	0.861±0.023	0.876±0.021	0.755±0.019	0.918±0.006	0.933±0.004	0.802±0.004	0.793±0.009	0.820±0.011	0.721±0.005	0.923±0.002	0.904±0.004	0.854±0.002	0.852±0.004	0.881±0.001	0.786±0.001
TCL	0.777±0.032	0.761±0.026	0.684±0.026	0.927±0.009	0.927±0.010	0.732±0.037	0.778±0.004	0.782±0.015	0.703±0.010	0.929±0.004	0.914±0.005	0.885±0.005	0.835±0.003	0.867±0.003	0.786±0.005
DyGFormer	0.706±0.015	0.655±0.019	0.654±0.002	0.885±0.008	0.898±0.004	0.727±0.018	0.822±0.011	0.859±0.009	0.756±0.010	0.930±0.009	0.910±0.006	**0.899**±0.012	0.813±0.010	0.845±0.008	0.810±0.007
ContraTGT	**0.998**±0.001	**0.998**±0.001	**0.980**±0.001	**0.955**±0.003	**0.959**±0.003	**0.897**±0.003	**0.902**±0.008	**0.904**±0.011	**0.826**±0.039	**0.934**±0.007	**0.919**±0.009	0.862±0.005	**0.861**±0.007	**0.894**±0.009	**0.822**±0.005

Algorithm 3 SHARPEN Function

Input: Vector $\mathbf{h}$, number of samples K, hyperparameter ϵ;
Output: Sharpened vector $\mathbf{f}$.

$\mathbf{m} \leftarrow \mathbf{0}$;
for $i \in \{1, \ldots, K\}$ **do**
  $\hat{\mathbf{h}} \leftarrow \mathbf{h}$;
  $\mathbf{w} \leftarrow 1 - \hat{\mathbf{h}}$;
  $\log \mathbf{w} \leftarrow \log(\mathbf{w} + \epsilon)$; ▷ Mask Top-(i-1) elements
  $\mathbf{y} \leftarrow \mathbf{h} + \log \mathbf{w}$;
  $\mathbf{y} \leftarrow \mathbf{y} - \max(\mathbf{h})$;
  $\mathbf{y} \leftarrow \text{SOFTMAX}(\mathbf{y})$; ▷ Integrate the mask with the input and sharpen the result.
  $\mathbf{m} \leftarrow \mathbf{m} + \mathbf{y} * \mathbf{w}$;
  $\mathbf{f} \leftarrow \mathbf{f} \cdot \mathbf{m}$; ▷ Update the output vector
return $\mathbf{f}$;

- **soc-sign-bitcoinotc**[7]: This is a signed trust network on a platform called Bitcoin OTC[8], where people trade using Bitcoin. It is noteworthy that no repeated interactions happen in this network. The network contains 5.8 thousand users and 35.5 thousand trust ratings.
- **Social Evolve 1_month** (53): Social Evolution records physical proximity between students living in the dormitory overtime. The original dataset spans one year, we split out the data over a month, termed Social Evolve 1m.
- **Ubuntu**[9]: This is a stack exchange recorded from the website Ask Ubuntu 8. Nodes are users and there are three different types of edges, (1) user u answering user v's question, (2) user u commenting on user v's question, and (3) user w commenting on user u's answer. The dataset contains more than 100K nodes.

2) Baselines: To comprehensively evaluate our proposed ContraTGT against state-of-the-art approaches, we consider several state-of-the-art baselines categorized into three main groups[10]: (1) *Static GNN models*: GraphSAGE (27); (2) *Temporal GNN models*: JOIDE (7), CAWN (37), TIP-GNN (54), TAP-GNN (55); (3) *Approaches with advanced techniques*: DyRep (38), TGAT (10), APAN (31), TGN (8), TIGER (56), and FTM (57); and (4) *Graph Transformer approaches*: TCL (16) and DyGFormer (58). We provide further illustrations of the baselines below.

- **GraphSAGE** (27): GraphSAGE learns node representation vectors by aggregating the neighbor information of each node, enabling classification or regression tasks.
- **JODIE** (7): JODIE employs a coupled RNN architecture and incorporates an attention layer to obtain the final representation of the user.
- **CAWN** (37): CAWN extracts network motifs based on random walk sampling and establishes correlations using a set based anonymization method.
- **TIP-GNN** (54): TIP-GNN models the believe graph structure to highlights nodes' personalized transition patterns among neighbors.
- **TAP-GNN** (55): TAP-GNN incorporates a time encoding kernel and an attention mechanism into dynamic node embeddings based on the sampled temporal subgraph.
- **DyRep** (38): DyRep models the dynamic processes of complex and nonlinear evolution and learns to encode the structural temporal information over graph.
- **TGAT** (10): TGAT proposes temporal graph attention layer, incorporates a time encoding kernel and attention mechanism to aggregate temporal topological neighborhood.
- **APAN** (31):Apan represents node temporal information using temporal context, extracts diverse semantic features from time intervals through attention mechanism, and encodes them into node embedding vectors.
- **TGN** (8): TGN is an extension of TGAT, which captures long-term interactions relations by adding a memory modules
- **TIGER** (56): TIGER introduce the dual memory to alleviate the staleness problem and improve the interpretability of memory.
- **FTM** (57): FTM preserves the short-term features and captures the intrinsic dynamics of graph evolution as long-term features.
- **TCL** (16): TCL employs two types of time encodings, including depth embedding and time embedding, to fuse with the context of the target node as the input of the graph Transformer.
- **DyGFormer** (58): DyGFormer is a Transformer-based dynamic graph learning architecture that introduces the DyGLib library to promote reproducible, scalable, and credible research in dynamic graph learning. By encoding neighbors, links, and time intervals, as well as neighbor co-occurrences, it resorts to a modified graph Transformer to model the temporal graph structure.

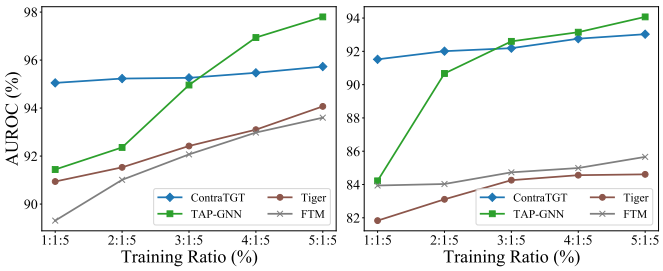
3) Settings and parameters: We evaluate using temporal link prediction. For every dataset, interactions are randomly divided in a 1:1:8 ratio for training, validation, and testing.

B. Performance Evaluation

In this part, we evaluate the performance of ContraTGT on the task of temporal link prediction on temporal graphs.

1) Evaluation with limited training data: In this experiment, we limit the training data to only 10% of the interactions to simulate scenarios with limited supervision. We conduct the experiments under

[7]https://networkrepository.com/soc-sign-bitcoinotc.php
[8]https://www.bitcoin-otc.com/
[9]https://snap.stanford.edu/data/sx-askubuntu.html
[10]For the temporal baselines, we only consider continuous-time models.



(a) ia-slashdot-reply-dir (b) soc-sign-bitcoinotc

Fig. 2: Evaluation with more training data.

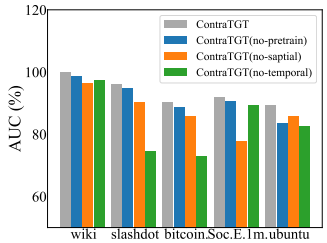


Fig. 3: Ablation study.

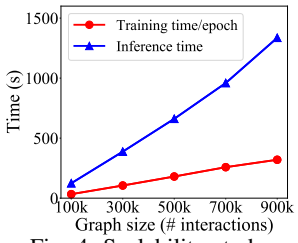


Fig. 4: Scalability study.

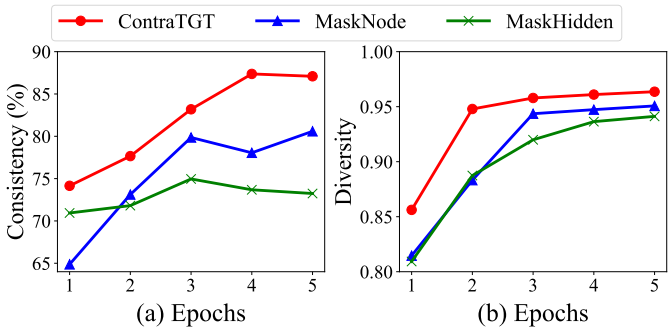


Fig. 5: Comparison with different augmentations w.r.t. consistency and diversity.

both transductive and inductive settings. In the transductive setting, the model is evaluated on the entire test dataset, while the inductive setting focuses on predicting interactions involving at least one new node that did not appear in the training data.

The performance comparison in Table II and Table III presents results for the transductive and inductive settings, respectively. Notably, ContraTGT consistently outperforms all the baselines across the three evaluation metrics, highlighting the efficacy of our dual-view representation learning and contrastive strategy, complemented by learnable data augmentation. An exception is the baseline DyGFormer, which occasionally achieves slightly better performance on the Social Evolve 1_month dataset. This phenomenon may be attributed to the dense interactions of this dataset and DyGFormer's utilization of the modified Transformer, which enhances its ability to capture local context effectively.

In addition, we also have the following additional observations. Firstly, static GNN models like GraphSAGE generally exhibit suboptimal performance since they lack specialized modules for addressing temporal patterns. Secondly, temporal GNN models and advanced techniques-based approaches do not consistently outperform each other. Temporal GNN models are equipped with tailored modules for temporal pattern recognition, resulting in competitive performance. On the other hand, approaches with advanced techniques display promise in temporal graph representation, especially in the inductive setting, suggesting their capacity to predict new data using well-designed mechanisms like attention or memory.

2) Evaluation with more training data: To further assess the efficacy of our proposed model with an increased volume of training data, we executed additional experiments, varying the quantity of training data while keeping the validation and test sets constant at a ratio of n:1:5, where $n = \{1, 2, 3, 4, 5\}$ represents the proportion of the training data. These experiments were carried out on two datasets (ia-slashdot-reply-dir and soc-sign-bitcoinotc), and we selected the most competitive baselines for comparison. The results, with AUC as metric in a transductive setting, are depicted in Fig. 2.

We have the following observations. (1) In scenarios with limited supervision, such as when the proportion of training data is 1 or 2, our proposed model, ContraTGT, surpasses the baselines, underscoring the strength of our model enhanced by pre-training with learnable data augmentation. (2) As the amount of training data increases, the baselines begin to demonstrate comparable, or in some cases, superior performance. This indicates that with ample training data at their disposal, end-to-end models like TAP-GNN, Tiger, and FTM are capable of fully leveraging label information to optimize their models. Typically, these models can surpass those that rely on pre-training when sufficient data is provided, at which point our model still maintains superior performance.

C. Model Analysis

We conduct a comprehensive analysis of various facets of ContraTGT, including an ablation study, scalability analysis, and a comparison of different augmentations. For a more thorough investigation, we have included additional experiments such as an extended comparison of various augmentations and an examination of parameter sensitivity.

1) Ablation study: To evaluate the contribution of each component in ContraTGT, we conduct an ablation study by comparing with several degenerate variants: (1) *without pretrain*: we use the ContraTGT architecture without the pre-training step; (2) *no spatial*: we remove the spatial-view sequences, by only using the temporal-view sequences for node representation learning. (3)*no temporal*: we remove the temporal-view sequences, by only using the spatial-view sequences for node representation learning.

We show the results of ablation study (using AUC as the metric) in Fig. 3, and make the following observations. Firstly, the performance consistently drops when the model pretraining step is omitted. This finding underscores the efficacy of the proposed contrastive learning strategy, which is significantly enhanced by the learnable data augmentation. Secondly, when the spatial sequences are omitted, ContraTGT exhibits a notable performance decline. This underscores the indispensable nature of spatial information for node representation learning within transformers applied to temporal graphs. Thirdly, the absence of temporal sequences also leads to a general performance decrease. This demonstrates the value of incorporating temporal sampling into ContraTGT. Notably, both spatial and temporal sequences seem to contribute comparably to the overall performance.

2) Scalability: To assess scalability via experiments, we utilized the largest temporal graph in our study, namely Ubuntu. We sampled several temporal graphs with increasing sizes, specifically 100k, 300k, 500k, 700k, and 900k interactions, to assess their training and inference times. For each temporal graph, we maintained the same data split as in our main experiment, with a 1:1:8 ratio for the training, validation, and testing sets. The training (per epoch) and inference times are reported in Fig. 4. Our observations reveal that both training

TABLE III: Inductive learning task results for link prediction.

Methods	soc-wiki-elec			ia-slashdot-reply-dir			soc-sign-bitcoinotc			Social E. 1 m.			ubuntu		
	AUC	AP	Acc	AUC	AP	Acc	AUC	AP	Acc	AUC	AP	Acc	AUC	AP	Acc
GraphSAGE	0.596±0.012	0.595±0.017	0.569±0.016	0.593±0.012	0.582±0.009	0.574±0.011	0.554±0.011	0.559±0.009	0.536±0.007	0.540±0.009	0.530±0.011	0.509±0.008	0.544±0.007	0.531±0.009	0.509±0.014
JODIE	0.548±0.009	0.550±0.007	0.532±0.010	0.638±0.017	0.641±0.016	0.608±0.012	0.679±0.016	0.692±0.020	0.633±0.022	0.818±0.003	0.755±0.006	0.761±0.005	0.762±0.007	0.796±0.008	0.682±0.005
CAWN	0.961±0.003	0.965±0.005	0.893±0.015	0.845±0.012	0.863±0.012	0.743±0.014	0.748±0.015	0.798±0.009	0.684±0.010	0.903±0.012	0.883±0.024	0.859±0.016	0.796±0.078	0.835±0.065	0.745±0.081
TIP-GNN	0.694±0.007	0.593±0.019	0.631±0.013	0.773±0.005	616±0.009	0.637±0.006	0.821±0.012	0.676±0.008	0.737±0.010	0.812±0.011	0.685±0.009	0.743±0.013	0.775±0.007	0.776±0.006	0.757±0.008
DyRep	0.532±0.016	0.538±0.012	0.523±0.011	0.633±0.009	0.639±0.016	0.603±0.008	0.689±0.006	0.697±0.008	0.636±0.020	0.716±0.039	0.687±0.038	0.651±0.042	0.746±0.002	0.780±0.004	0.685±0.010
TGAT	0.785±0.014	0.761±0.019	0.699±0.023	0.823±0.004	0.869±0.002	0.776±0.005	0.790±0.007	0.714±0.016	0.620±0.012	0.630±0.012	0.559±0.004	0.806±0.027	0.805±0.027	0.844±0.018	0.747±0.028
APAN	0.832±0.009	0.844±0.006	0.797±0.006	0.784±0.004	0.775±0.004	0.707±0.007	0.774±0.005	0.768±0.006	0.721±02	0.752±0.005	0.738±0.008	0.718±0.007	0.726±0.007	0.776±0.004	0.649±0.008
TGN	0.886±0.008	0.896±0.007	0.802±0.056	0.867±0.017	0.880±0.016	0.692±0.007	0.773±0.006	0.809±0.006	0.701±0.006	0.909±0.007	0.881±0.011	0.852±0.005	0.824±0.007	0.859±0.004	0.763±0.004
TIGER	0.832±0.010	0.845±0.012	0.696±0.008	0.939±0.05	0.947±0.003	0.804±0.007	0.763±0.014	0.806±0.008	0.694±0.016	**0.952**±0.002	**0.938**±0.004	**0.918**±0.007	0.826±0.004	0.878±0.009	0.609±0.014
FTM	0.878±0.017	0.888±0.015	0.760±0.015	0.919±0.006	0.934±0.004	0.807±0.002	0.799±0.011	0.826±0.012	0.728±0.004	0.928±0.003	0.908±0.001	0.849±0.003	0.846±0.004	0.862±0.019	0.779±0.001
ContraTGT	**0.997**±0.001	**0.998**±0.001	**0.981**±0.001	**0.955**±0.003	**0.960**±0.003	**0.895**±0.004	**0.899**±0.008	**0.901**±0.012	**0.824**±0.040	0.943±0.007	0.935±0.010	0.877±0.008	**0.857**±0.007	**0.892**±0.010	**0.820**±0.0058

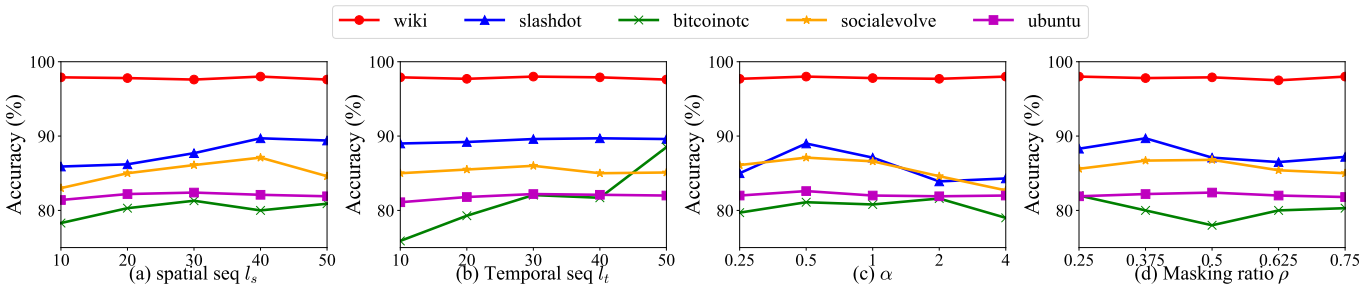


Fig. 6: Influence of parameters.

TABLE IV: Compare with different augmentations.

Methods	slashdot			Social E. 1m		
	AUC	AP	Acc	AUC	AP	Acc
MaskNode	0.944±0.003	0.947±0.002	0.866±0.005	0.929±0.003	0.912±0.005	0.853±0.005
MaskHidden	0.937±0.001	0.955±0.006	0.881±0.008	0.914±0.010	0.897±0.006	**0.889**±0.006
ContraTGT	**0.954**±0.003	**0.956**±0.002	**0.891**±0.004	**0.938**±0.004	**0.924**±0.004	0.877±0.006

and inference times for our proposed ContraTGT, scale linearly with the increase in temporal graph size. This linear scalability underscores ContraTGT's capability to be efficiently applied to large-scale temporal graphs, aligning with the complexity analysis provided in Section IV-D.

3) Comparison of different augmentations: In contrast to random masking methods, our proposed augmentation method excels at adaptively selecting the most informative nodes in the sequence for representation learning, guided by the self-supervised consistency and diversity objective.

To assess the effectiveness of our proposed learnable data augmentation method, integrated with the graph transformer featuring dual-view sequence sampling, we introduce two simple yet efficient random masking methods: MaskNode and MaskHidden. For MaskNode, we randomly mask a few nodes in a sampled node sequence according to the predefined masking rate ρ and then compute its representation. Meanwhile, for MaskHidden, we first compute the representation of the entire sequence and then randomly mask a few dimensions with respect to the predefined masking rate ρ, akin to the dropout mechanism. We compare the performance of these three approaches on two representative datasets, presenting the results in Table IV. Note that, for each dataset, we tune the masking rate ρ in the range of [0.1, 0.7] for each method to achieve optimal performance. We observe that our proposed model generally outperforms the other two random masking methods. This observation underscores the capability of our learnable data augmentation to effectively leverage underlying structures for self-supervised representation learning, surpassing the performance achieved by random masking methods.

We also conducted an experiment to compare different data augmentation methods, using consistency and diversity as the evaluation metrics for the generated data augmentations. Note that for both metrics, higher values indicate better performance. We compared our proposed learnable data augmentation method with MaskNode and MaskHidden approaches on one dataset, *i.e.*, Bitcoinotc, and present the results in Fig. 5. We observe that, as training advances, it becomes evident that our model persistently surpasses the random masking techniques, namely MaskNode and MaskHidden, securing markedly enhanced consistency and diversity. This trend highlights the efficacy of our learnable data augmentation strategy, which evidently produces more advantageous data augmentations when contrasted with random masking methods. The superiority of these data augmentations plays a pivotal role in amplifying the success of our contrastive learning approach. Moreover, the progressive improvement of our proposed model with each epoch is noteworthy. In contrast, the performance trend of the random masking approaches appears more variable, particularly in terms of consistency.

4) Parameters Sensitivity: We evaluate the sensitivity of several important hyperparameters in ContraTGT, and show their impact in Fig. 6. For the length of spatial and temporal sequences, *i.e.*, l_s and l_t, the performance seems to be stable w.r.t. the length, while longer sequences might be slightly better. For the weight of α in Eq. (13), it seems that a smaller α can benefit the performance, showing that consistency is more important in determining the performance. For the masking ratio ρ, the performance seems to be stable across different ρ's. In particular, $\rho \in [0.25, 0.50]$ might be a good choice.

Please note that the performance of hyperparameters may occasionally exhibit some degree of smoothness. The underlying reasons for this phenomenon are three-fold. (1) Regarding the length of spatial and temporal sequences, the consistency and diversity metrics proposed in our work play a crucial role in guiding the generation of informative augmentations. The effectiveness of our learnable data augmentation, driven by these metrics, ensures that the model is pre-trained with the most informative augmentations even with different l_s and l_t, contributing to the smooth performance across varying sequence lengths. (2) The smoothness in the masking ratio may be attributed to the effectiveness of the learnable data augmentation mechanism. This mechanism is designed to selectively choose the most informative elements for augmentation, even under extreme

masking ratios, leading to promising augmentation generation. (3) For the wiki dataset, the graph Transformer backbone armed with dual-view sequences achieves nearly 100% performance, leaving limited room for significant improvement. As for the Ubuntu dataset, its relatively large size and the utilization of 10% interactions for training may already provide sufficient data for training the graph Transformer. In such cases, the pre-training procedure may contribute less to the overall performance. In summary, our proposed learnable data augmentation could achieve both higher consistency and diversity compared to MaskNode and MaskHidden.

VI. Conclusions

This paper delves into the under-explored domain of contrastive learning of temporal graph transformers. Specifically, we present a novel model, ContraTGT, which employs a dual-view graph transformer to extract sequences tailored for specific target nodes, encapsulating both spatial and temporal perspectives. To optimize the contrastive learning of this dual-view transformer, we put forth an innovative, learnable data augmentation method, which involves selective masking of elements within dual-view sequences, generates superior augmentations, thereby amplifying the potency of the contrastive learning approach. Extensive experiments on five public temporal network datasets demonstrate that our model can consistently outperform all baselines, especially in small training set conditions.

References

[1] S. M. Kazemi, R. Goel, K. Jain, I. Kobyzev, A. Sethi, P. Forsyth, and P. Poupart, "Representation learning for dynamic graphs: A survey," *The Journal of Machine Learning Research*, vol. 21, no. 1, pp. 2648–2720, 2020.

[2] C. D. Barros, M. R. Mendonça, A. B. Vieira, and A. Ziviani, "A survey on embedding dynamic graphs," *ACM Computing Surveys (CSUR)*, vol. 55, no. 1, pp. 1–37, 2021.

[3] G. H. Nguyen, J. B. Lee, R. A. Rossi, N. K. Ahmed, E. Koh, and S. Kim, "Continuous-time dynamic network embeddings," in *Companion proceedings of the the web conference 2018*, 2018, pp. 969–976.

[4] Y. Zhang, Y. Xiong, X. Kong, and Y. Zhu, "Learning node embeddings in interaction graphs," in *Proceedings of the 2017 ACM on Conference on Information and Knowledge Management*, 2017, pp. 397–406.

[5] A. Longa, V. Lachi, G. Santin, M. Bianchini, B. Lepri, P. Lio, F. Scarselli, and A. Passerini, "Graph neural networks for temporal graphs: State of the art, open challenges, and opportunities," *arXiv preprint arXiv:2302.01018*, 2023.

[6] Z. A. Sahili and M. Awad, "Spatio-temporal graph neural networks: A survey," *arXiv preprint arXiv:2301.10569*, 2023.

[7] S. Kumar, X. Zhang, and J. Leskovec, "Predicting dynamic embedding trajectory in temporal interaction networks," in *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*, 2019, pp. 1269–1278.

[8] E. Rossi, B. Chamberlain, F. Frasca, D. Eynard, F. Monti, and M. Bronstein, "Temporal graph networks for deep learning on dynamic graphs," *arXiv preprint arXiv:2006.10637*, 2020.

[9] A. Pareja, G. Domeniconi, J. Chen, T. Ma, T. Suzumura, H. Kanezashi, T. Kaler, T. Schardl, and C. Leiserson, "Evolvegcn: Evolving graph convolutional networks for dynamic graphs," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 34, no. 04, 2020, pp. 5363–5370.

[10] da Xu, chuanwei ruan, evren korpeoglu, sushant kumar, and kannan achan, "Inductive representation learning on temporal graphs," in *International Conference on Learning Representations*, 2020.

[11] Q. Zhang, C. Huang, L. Xia, Z. Wang, S. M. Yiu, and R. Han, "Spatial-temporal graph learning with adversarial contrastive adaptation," in *International Conference on Machine Learning*. PMLR, 2023, pp. 41 151–41 163.

[12] Y. Zhu, F. Cong, D. Zhang, W. Gong, Q. Lin, W. Feng, Y. Dong, and J. Tang, "Wingnn: Dynamic graph neural networks with random gradient aggregation window," in *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2023, pp. 3650–3662.

[13] C. Ying, T. Cai, S. Luo, S. Zheng, G. Ke, D. He, Y. Shen, and T.-Y. Liu, "Do transformers really perform badly for graph representation?" *Advances in Neural Information Processing Systems*, vol. 34, pp. 28 877–28 888, 2021.

[14] D. Kreuzer, D. Beaini, W. Hamilton, V. Létourneau, and P. Tossou, "Rethinking graph transformers with spectral attention," *Advances in Neural Information Processing Systems*, vol. 34, pp. 21 618–21 629, 2021.

[15] V. P. Dwivedi and X. Bresson, "A generalization of transformer networks to graphs," *arXiv preprint arXiv:2012.09699*, 2020.

[16] L. Wang, X. Chang, S. Li, Y. Chu, H. Li, W. Zhang, X. He, L. Song, J. Zhou, and H. Yang, "Tcl: Transformer-based dynamic graph modelling via contrastive learning," *arXiv preprint arXiv:2105.07944*, 2021.

[17] W. Cong, Y. Wu, Y. Tian, M. Gu, Y. Xia, C.-c. J. Chen, and

M. Mahdavi, "Dyformer: A scalable dynamic graph transformer with provable benefits on generalization ability," in *Proceedings of the 2023 SIAM International Conference on Data Mining (SDM)*. SIAM, 2023, pp. 442–450.

[18] W. Hu, B. Liu, J. Gomes, M. Zitnik, P. Liang, V. Pande, and J. Leskovec, "Strategies for pre-training graph neural networks," *arXiv preprint arXiv:1905.12265*, 2019.

[19] Z. Hu, Y. Dong, K. Wang, K.-W. Chang, and Y. Sun, "Gpt-gnn: Generative pre-training of graph neural networks," in *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2020, pp. 1857–1867.

[20] J. Qiu, Q. Chen, Y. Dong, J. Zhang, H. Yang, M. Ding, K. Wang, and J. Tang, "Gcc: Graph contrastive coding for graph neural network pre-training," in *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*, 2020, pp. 1150–1160.

[21] Y. You, T. Chen, Y. Sui, T. Chen, Z. Wang, and Y. Shen, "Graph contrastive learning with augmentations," *Advances in neural information processing systems*, vol. 33, pp. 5812–5823, 2020.

[22] Y. Zhu, Y. Xu, F. Yu, Q. Liu, S. Wu, and L. Wang, "Deep graph contrastive representation learning," *arXiv preprint arXiv:2006.04131*, 2020.

[23] Y. Rong, W. Huang, T. Xu, and J. Huang, "Dropedge: Towards deep graph convolutional networks on node classification," in *8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020*. OpenReview.net, 2020. [Online]. Available: https://openreview.net/forum?id=Hkx1qkrKPr

[24] W. Feng, J. Zhang, Y. Dong, Y. Han, H. Luan, Q. Xu, Q. Yang, E. Kharlamov, and J. Tang, "Graph random neural networks for semi-supervised learning on graphs," in *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, Eds., vol. 33. Curran Associates, Inc., 2020, pp. 22 092–22 103.

[25] A. Grover and J. Leskovec, "node2vec: Scalable feature learning for networks," in *Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining*, 2016, pp. 855–864.

[26] J. Tang, M. Qu, M. Wang, M. Zhang, J. Yan, and Q. Mei, "Line: Large-scale information network embedding," in *Proceedings of the 24th international conference on world wide web*, 2015, pp. 1067–1077.

[27] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," *Advances in neural information processing systems*, vol. 30, 2017.

[28] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *International Conference on Learning Representations*, 2018.

[29] L. Zhou, Y. Yang, X. Ren, F. Wu, and Y. Zhuang, "Dynamic network embedding by modeling triadic closure process," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 32, no. 1, 2018.

[30] Z. Zhang, J. Bu, M. Ester, J. Zhang, C. Yao, Z. Li, and C. Wang, "Learning temporal interaction graph embedding via coupled memory networks," in *Proceedings of the web conference 2020*, 2020, pp. 3049–3055.

[31] X. Wang, D. Lyu, M. Li, Y. Xia, Q. Yang, X. Wang, X. Wang, P. Cui, Y. Yang, B. Sun *et al.*, "Apan: Asynchronous propagation attention network for real-time temporal graph embedding," in *Proceedings of the 2021 international conference on management of data*, 2021, pp. 2628–2638.

[32] L. Qu, H. Zhu, Q. Duan, and Y. Shi, "Continuous-time link prediction via temporal dependent graph neural network," in *Proceedings of The Web Conference 2020*, 2020, pp. 3026–3032.

[33] A. Sankar, Y. Wu, L. Gou, W. Zhang, and H. Yang, "Dysat: Deep neural representation learning on dynamic graphs via self-attention networks," in *Proceedings of the 13th international conference on web search and data mining*, 2020, pp. 519–527.

[34] K. Zhang, Q. Cao, G. Fang, B. Xu, H. Zou, H. Shen, and X. Cheng, "Dyted: Disentangled representation learning for discrete-time dynamic graph," in *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2023, pp. 3309–3320.

[35] Q. Bai, C. Nie, H. Zhang, D. Zhao, and X. Yuan, "Hgwavenet: A hyperbolic graph neural network for temporal link prediction," in *Proceedings of the ACM Web Conference 2023*, 2023, pp. 523–532.

[36] S. Zhang, Y. Xiong, Y. Zhang, Y. Sun, X. Chen, Y. Jiao, and Y. Zhu, "Rdgsl: Dynamic graph representation learning with structure learning," *arXiv preprint arXiv:2309.02025*, 2023.

[37] Y. Wang, Y.-Y. Chang, Y. Liu, J. Leskovec, and P. Li, "Inductive representation learning in temporal networks via causal anonymous walks," *arXiv preprint arXiv:2101.05974*, 2021.

[38] R. Trivedi, M. Farajtabar, P. Biswal, and H. Zha, "Dyrep: Learning representations over dynamic graphs," in *International Conference on Learning Representations*, 2019. [Online]. Available: https://openreview.net/forum?id=HyePrhR5KX

[39] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.

[40] J. Zhao, C. Li, Q. Wen, Y. Wang, Y. Liu, H. Sun, X. Xie, and Y. Ye, "Gophormer: Ego-graph transformer for node classification," *arXiv preprint arXiv:2110.13094*, 2021.

[41] F. Poursafaei, S. Huang, K. Pelrine, and R. Rabbany, "Towards better evaluation for dynamic link prediction," *Advances in Neural Information Processing Systems*, vol. 35, pp. 32 928–32 941, 2022.

[42] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A simple framework for contrastive learning of visual representations," in *International conference on machine learning*. PMLR, 2020, pp. 1597–1607.

[43] H. Zhang, X. Han, X. Xiao, and J. Bai, "Time-aware graph structure learning via sequence prediction on temporal graphs," *arXiv preprint arXiv:2306.07699*, 2023.

[44] D. Bo, B. Hu, X. Wang, Z. Zhang, C. Shi, and J. Zhou, "Regularizing graph neural networks via consistency-diversity graph augmentations," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 36, no. 4, 2022, pp. 3913–3921.

[45] Y. You, T. Chen, Y. Shen, and Z. Wang, "Graph contrastive learning automated," in *International Conference on Machine Learning*. PMLR, 2021, pp. 12 121–12 132.

[46] Y. Zhu, Y. Xu, F. Yu, Q. Liu, S. Wu, and L. Wang, "Graph contrastive learning with adaptive augmentation," in *Proceedings of the Web Conference 2021*, 2021, pp. 2069–2080.

[47] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer normalization," *arXiv preprint arXiv:1607.06450*, 2016.

[48] Z. Wu, P. Jain, M. Wright, A. Mirhoseini, J. E. Gonzalez, and I. Stoica, "Representing long-range context for graph neural networks with global attention," *Advances in Neural Information Processing Systems*, vol. 34, pp. 13 266–13 279, 2021.

[49] Y. Rong, Y. Bian, T. Xu, W. Xie, Y. Wei, W. Huang, and J. Huang, "Self-supervised graph transformer on large-scale molecular data," *Advances in Neural Information Processing Systems*, vol. 33, pp. 12 559–12 571, 2020.

[50] E. Min, R. Chen, Y. Bian, T. Xu, K. Zhao, W. Huang, P. Zhao,

J. Huang, S. Ananiadou, and Y. Rong, "Transformer for graphs: An overview from architecture perspective," *arXiv preprint arXiv:2202.08455*, 2022.

[51] Q. Mao, Z. Liu, C. Liu, and J. Sun, "Hinormer: Representation learning on heterogeneous information networks with graph transformer," in *Proceedings of the ACM Web Conference 2023*, 2023, pp. 599–610.

[52] Z. Li, X. Wang, Y. Huang, and M. Zhang, "Is distance matrix enough for geometric deep learning?" *Advances in Neural Information Processing Systems*, vol. 36, 2024.

[53] A. Madan, M. Cebrian, S. Moturu, K. Farrahi *et al.*, "Sensing the" health state" of a community," *IEEE Pervasive Computing*, vol. 11, no. 4, pp. 36–45, 2011.

[54] T. Zheng, Z. Feng, T. Zhang, Y. Hao, M. Song, X. Wang, X. Wang, J. Zhao, and C. Chen, "Transition propagation graph neural networks for temporal networks," *IEEE Transactions on Neural Networks and Learning Systems*, 2022.

[55] T. Zheng, X. Wang, Z. Feng, J. Song, Y. Hao, M. Song, X. Wang, X. Wang, and C. Chen, "Temporal aggregation and propagation graph neural networks for dynamic representation," *IEEE Transactions on Knowledge and Data Engineering*, 2023.

[56] Y. Zhang, Y. Xiong, Y. Liao, Y. Sun, Y. Jin, X. Zheng, and Y. Zhu, "Tiger: Temporal interaction graph embedding with restarts," *arXiv preprint arXiv:2302.06057*, 2023.

[57] B. Cao, Q. Ye, W. Xu, and Y. Zou, "Ftm: A frame-level timeline modeling method for temporal graph representation learning," *arXiv preprint arXiv:2302.11814*, 2023.

[58] L. Yu, L. Sun, B. Du, and W. Lv, "Towards better dynamic graph learning: New architecture and unified library," *NeurIPS*, 2023.